

ESPECIFICAÇÃO COMPLETA DA LINGUAGEM MINIC

Contrato de linguagem para o projeto de compiladores

Versão 1.0 — Documento acadêmico

1. Objetivo e escopo

MINIC é uma linguagem educacional inspirada em um subconjunto de C, projetada para a construção incremental de um compilador. Esta especificação funciona como contrato comum entre as quatro etapas do projeto: análise léxica; análise sintática e AST; análise semântica e representação intermediária; geração de código e otimização.

A versão mínima contempla variáveis tipadas, funções, escopo léxico, vetores unidimensionais, estruturas condicionais e de repetição, operadores aritméticos, relacionais e lógicos, entrada e saída simples e comentários.

1.1 Componentes da implementação

Componente	Responsabilidade	Saída principal
Lexer	Converter caracteres em tokens.	Sequência de tokens com linha e coluna.
Parser	Validar a gramática e construir a AST.	AST ou diagnóstico sintático.
Analizador semântico	Verificar nomes, escopos, tipos e regras contextuais.	AST anotada ou diagnósticos.
Gerador de IR	Traduzir a AST para código de três endereços.	IR estruturada.
Otimizador	Aplicar transformações corretas à IR.	IR otimizada.
Gerador de código	Produzir código de destino executável.	Assembly, LLVM IR ou máquina virtual.

2. Modelo de programa

Todo programa MINIC deve conter exatamente uma função main. A forma recomendada é int main(), com retorno inteiro.

```
int main() {
    return 0;
}
```

A estrutura geral recomendada é: declarações globais opcionais, declarações de funções e função main. Funções e variáveis globais devem ser declaradas antes de seu uso, salvo se a implementação adotar declarações antecipadas explicitamente documentadas.

```
int limite = 10;

int dobro(int valor) {
    return valor * 2;
}

int main() {
    print(dobro(limite));
    return 0;
}
```

3. Especificação léxica

3.1 Convenções gerais

- A linguagem diferencia maiúsculas de minúsculas.
- Espaços, tabulações e quebras de linha são ignorados fora de literais.
- O lexer deve preservar linha e coluna para diagnósticos.

3.2 Comentários

Comentários de linha iniciam com `//` e terminam na quebra de linha. Comentários de bloco iniciam com `/*` e terminam com `*/`. Comentários não podem ser aninhados.

3.3 Palavras reservadas

Palavra	Finalidade
<code>int</code>	Inteiro
<code>float</code>	Real
<code>bool</code>	Booleano
<code>char</code>	Caractere
<code>void</code>	Sem retorno
<code>if / else</code>	Condicional
<code>while / for</code>	Repetição
<code>return</code>	Retorno
<code>break / continue</code>	Controle de laços
<code>true / false</code>	Literais booleanos
<code>print / read</code>	Saída e entrada

3.4 Tokens obrigatórios

Categoria	Expressão ou exemplos
Identificador	<code>[A-Za-z_][A-Za-z0-9_]*</code>
Inteiro	<code>[0-9]+</code>
Real	<code>[0-9]+\.[0-9]+</code>
Caractere	<code>'a', '\n'</code>
Cadeia	<code>"texto"</code>
Aritméticos	<code>+ - * / %</code>
Relacionais	<code>== != < > <= >=</code>
Lógicos	<code>&& !</code>
Atribuição	<code>=</code>
Delimitadores	<code>() [] {} ; ,</code>

3.5 Identificadores e literais

Identificadores começam por letra ou sublinhado e podem continuar com letras, dígitos e sublinhados. Inteiros usam uma sequência de dígitos; reais usam parte inteira e decimal. Literais booleanos são true e false. Caracteres usam aspas simples e cadeias usam aspas duplas.

Escape	Significado
\n	Nova linha
\t	Tabulação
\\	Barra invertida
\'	Aspas simples
\"	Aspas duplas

4. Gramática sintática (EBNF)

A notação a seguir é uma EBNF simplificada. Símbolos entre aspas são terminais; o sufixo ? indica opcionalidade; * indica repetição; + indica uma ou mais ocorrências.

```

programa ::= declaracao_global* declaracao_funcao* funcao_main ;
declaracao_global ::= tipo identificador inicializacao? ";" ;
declaracao_funcao ::= tipo_retorno identificador "(" parametros? ")" bloco ;
tipo_retorno ::= tipo | "void" ;
parametros ::= parametro ("," parametro)* ;
parametro ::= tipo identificador | tipo identificador "[" tamanho "]" ;
tipo ::= "int" | "float" | "bool" | "char" ;
bloco ::= "{" item_bloco* "}" ;
item_bloco ::= declaracao_local | comando ;
declaracao_local ::= tipo declarador ("," declarador)* ";" ;
declarador ::= identificador inicializacao? | identificador "[" tamanho "]" ;
inicializacao ::= "=" expressao ;

```

4.1 Comandos

```

comando ::= comando_expressao
| comando_bloco
| comando_if
| comando_while
| comando_for
| comando_return
| comando_break
| comando_continue
| comando_print
| comando_read ;
comando_expressao ::= expressao? ";" ;
comando_if ::= "if" "(" expressao ")" comando ("else" comando)? ;

```

```

comando_while ::= "while" "(" expressao ")" comando ;
comando_for ::= "for" "(" expressao? ";" expressao? ";" expressao? ")" comando ;
comando_return ::= "return" expressao? ";" ; comando_break ::= "break" ";" ;
comando_continue ::= "continue" ";" ;
comando_print ::= "print" "(" argumento_print ")" ";" ;
comando_read ::= "read" "(" localizavel ")" ";" ;

```

4.2 Expressões

```

expressao ::= atribuicao ;
atribuicao ::= localizavel "=" atribuicao | expressao_or ;
expressao_or ::= expressao_and | expressao_or "||" expressao_and ;
expressao_and ::= expressao_igualdade | expressao_and "&&" expressao_igualdade ;
expressao_igualdade ::= expressao_relacional | expressao_igualdade ("==" | "!=")
expressao_relacional ;
expressao_relacional ::= expressao_aditiva | expressao_relacional op_rel
expressao_aditiva ;
op_rel ::= "<" | ">" | "<=" | ">=" ;
expressao_aditiva ::= expressao_multiplicativa | expressao_aditiva ("+" | "-")
expressao_multiplicativa ;
expressao_multiplicativa ::= expressao_unaria | expressao_multiplicativa ("*" | "/"
| "%") expressao_unaria ;
expressao_unaria ::= ("-" | "!") expressao_unaria | expressao_posfixa ;
expressao_posfixa ::= primario | expressao_posfixa "[" expressao "]" |
expressao_posfixa "(" argumentos? ")" ;
primario ::= identificador | literal_inteiro | literal_real | literal_booleano |
literal_caractere | "(" expressao ")" ;

```

4.3 Precedência e associatividade

Nível	Operadores	Associatividade
1	() [] chamada	Esquerda
2	- !	Direita
3	* / %	Esquerda
4	+ -	Esquerda
5	< > <= >=	Esquerda
6	== !=	Esquerda
7	&&	Esquerda
8		Esquerda
9	=	Direita

5. Tipos e regras de conversão

Tipo	Descrição	Literais
int	Inteiro com sinal	0, 10, 42
float	Número real	3.14, 0.5
bool	Valor lógico	true, false
char	Um caractere	'A', '\n'
void	Ausência de retorno	Somente funções

Conversões implícitas recomendadas: int para float e char para int. Conversões float para int, bool para int, int para bool e float para bool não são implícitas na versão estrita. O operador de resto aceita apenas inteiros. Operadores relacionais retornam bool; operadores lógicos exigem operandos bool.

6. Escopos, símbolos e funções

A linguagem utiliza escopo léxico. Um bloco cria escopo próprio; variáveis locais ocultam declarações externas com o mesmo nome, conforme a política de sombreamento adotada. Não é permitida declaração duplicada no mesmo escopo.

6.1 Informação mínima da tabela de símbolos

- Nome, categoria e tipo.
- Escopo e nível léxico.
- Linha e coluna da declaração.
- Estado de inicialização e uso.
- Deslocamento no registro de ativação, quando aplicável.

Categorias possíveis:

- VARIÁVEL
- CONSTANTE
- FUNÇÃO
- PARÂMETRO
- VETOR

6.2 Operações da tabela de símbolos

```
inserir(nome, informacao)
buscar(nome)
buscar_no_escopo_atual(nome)
abrir_escopo()
fechar_escopo()
```

6.3 Funções

Funções possuem tipo de retorno, nome, parâmetros e corpo. O número e os tipos dos argumentos devem ser compatíveis. Funções não void devem retornar valor em todos os caminhos analisados. A recursão é permitida.

```
int fatorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * fatorial(n - 1);
}
```

7. Regras semânticas

Regra	Diagnóstico esperado
Nome	Uso de identificador não declarado é erro.
Declaração	Duplicidade no mesmo escopo é erro.
Atribuição	Lado esquerdo deve ser gravável e compatível.
Condição	if, while e for devem receber bool na versão estrita.
Função	Quantidade e tipos dos argumentos devem coincidir.
Retorno	Tipo retornado deve ser compatível com a assinatura.
Laços	break e continue só podem ocorrer dentro de laços.
Vetores	Índice deve ser int; limites devem ser tratados.
Divisão	Divisor constante zero deve ser rejeitado.

8. AST — árvore sintática abstrata

A AST elimina detalhes sintáticos que não são necessários às fases posteriores. Cada nó deve conter, sempre que possível, posição de origem, tipo inferido e referências a símbolos.

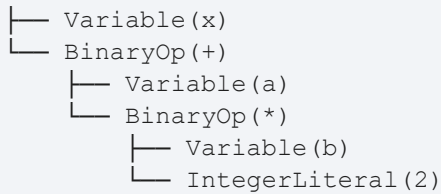
```
Program
├── GlobalDeclarations
├── FunctionDeclarations
└── MainFunction

FunctionDecl(returnType, name, parameters, body)
VarDecl(type, name, initializer?)
BinaryOp(operator, left, right)
UnaryOp(operator, operand)
IfStmt(condition, thenBranch, elseBranch?)
WhileStmt(condition, body)
CallExpr(function, arguments)
ReturnStmt(expression?)
```

8.1 Exemplo de AST

Código: `x = a + b * 2;`

Assignment



9. Representação intermediária

A IR recomendada é o código de três endereços (3AC), no qual cada instrução possui no máximo um resultado e dois operandos. A implementação pode armazenar instruções como quádruplas, triplas ou objetos estruturados.

9.1 Operações mínimas

```

ADD
SUB
MUL
DIV
MOD
NEG
NOT
ASSIGN
LOAD
STORE
LABEL
JUMP
JUMP_IF_FALSE
JUMP_IF_TRUE
PARAM
CALL
RETURN
  
```

9.2 Exemplos

Código:

```

x = a + b * c;
t1 = b * c
t2 = a + t1
x = t2
  
```

Código:

```

if (x < 10) {
  y = 1;
} else {
  y = 2;
}
t1 = x < 10

JUMP_IF_FALSE t1, L_else y = 1 JUMP L_end LABEL L_else y = 2 LABEL L_end
  
```

10. Geração de código e otimização

O projeto pode gerar assembly de uma máquina de pilha, assembly x86-64, LLVM IR ou código para uma máquina virtual própria. A escolha deve ser registrada no relatório e mantida estável entre as etapas.

10.1 Otimizações mínimas

- Uma otimização local, como simplificação algébrica, propagação de constantes ou eliminação de código morto.
- Uma otimização global, como reaching definitions, live variables ou eliminação de subexpressões comuns.
- A corretude deve ser demonstrada com testes antes e depois da otimização.

11. Interface de linha de comando

Comando	Resultado
minic programa.mc	Compila o programa.
minic --tokens programa.mc	Exibe tokens.
minic --ast programa.mc	Exibe a AST.
minic --symbols programa.mc	Exibe a tabela de símbolos.
minic --ir programa.mc	Exibe a IR.
minic --check programa.mc	Executa análises sem gerar código.
minic --target=asm programa.mc	Gera assembly.
minic -O0 / -O1 programa.mc	Desativa/ativa otimizações.

11.1 Códigos de saída

Código	Significado
0	Sucesso
1	Uso inválido da ferramenta
2	Erro léxico
3	Erro sintático
4	Erro semântico
5	Erro na geração de código
6	Erro durante execução

12. Diagnósticos e tratamento de erros

Todo diagnóstico deve informar categoria, linha, coluna, lexema ou token relevante e, quando possível, uma sugestão de correção. O parser deve tentar recuperação por modo pânico ou sincronização para reportar mais de um erro por execução, sem mascarar a causa original.

Erro léxico na linha 3, coluna 12:

símbolo "@" não reconhecido.

Erro sintático na linha 4, coluna 8:

esperado ")" após a condição; encontrado "{".

Erro semântico na linha 6, coluna 5:

identificador "resultado" não foi declarado.

13. Testes de conformidade

13.1 Programa válido completo

```
int soma(int a, int b) {
    return a + b;
}

int main() {
    int valores[3];
    int i;
    int total = 0;
    for (i = 0; i < 3; i = i + 1) {
        valores[i] = i * 2;
        total = total + valores[i];
    }
    print(soma(total, 1));
    return 0;
}
```

13.2 Casos inválidos obrigatórios

- Símbolo não reconhecido, literal não terminado e caractere com tamanho inválido.
- Parêntese, chave ou ponto e vírgula ausente.
- Variável não declarada e declaração duplicada.
- Tipos incompatíveis e chamada com número incorreto de argumentos.
- break fora de laço, retorno incompatível e acesso de vetor com índice não inteiro.

14. Organização recomendada do projeto

```
Minic/
├── README.md
├── docs/
│   ├── especificacao.md
│   ├── gramatica.ebnf
│   └── arquitetura.md
├── src/
│   ├── lexer/
│   ├── parser/
│   ├── ast/
│   ├── semantic/
│   ├── ir/
│   ├── optimizer/
│   └── codegen/
├── tests/
├── examples/
└── Makefile
```

15. Etapas de implementação e entregas

Etapa	Componente	Entregáveis
1	Analizador léxico	Tokens, expressões regulares, scanner, erros e testes.
2	Parser e AST	Gramática, parser, AST, recuperação e testes sintáticos.
3	Semântica e IR	Tabela de símbolos, escopos, tipos, 3AC e testes.
4	Código e otimização	Backend, duas otimizações, benchmarks, manual e apresentação.

16. Critérios de conformidade

Uma implementação é compatível com a especificação mínima quando reconhece os tokens e construções obrigatórias, ignora corretamente comentários e espaços, constrói AST, controla escopos, verifica tipos, valida funções, gera IR, produz código para os recursos implementados, emite diagnósticos com linha e coluna e possui testes automatizados.

Recursos opcionais devem ser identificados claramente e não podem alterar silenciosamente o significado dos programas da versão mínima.

17. Extensões possíveis

- Tipo string e biblioteca padrão.
- const, ++, -- e atribuições compostas.
- switch, do while, structs, ponteiros e matrizes multidimensionais.
- Conversões explícitas, inferência de tipos e funções genéricas.
- LLVM IR, JIT e otimizações avançadas.

Cada extensão deve documentar novos tokens, alterações na gramática, regras semânticas, AST, IR, geração de código e casos de teste.

18. Resumo da versão mínima

Área	Recursos obrigatórios
Tipos	int, float, bool, char, void
Comandos	declaração, atribuição, bloco, if/else, while, for, return, break, continue, print, read
Expressões	literais, identificadores, chamadas, vetores e operadores aritméticos, relacionais e lógicos
Estruturas	globais, locais, funções, parâmetros, vetores e escopos aninhados
Compilador	lexer, parser, AST, tabela de símbolos, semântica, IR, otimização e geração de código

19. Programas de referência

19.1 Fatorial recursivo

```
int fatorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * fatorial(n - 1);
}

int main() {
    print(fatorial(5));
    return 0;
}
```

19.2 Entrada, vetor e repetição

```
int main() {
    int valores[5];
    int i;
    int soma = 0;
    for (i = 0; i < 5; i = i + 1) {
        read(valores[i]);
        soma = soma + valores[i];
    }
    print(soma);
    return 0;
}
```

20. Controle de versões da especificação

Alterações incompatíveis devem gerar nova versão principal. Alterações compatíveis, correções editoriais e esclarecimentos podem incrementar a versão secundária. Toda implementação deve registrar quais recursos são obrigatórios, opcionais ou ainda não suportados.

Versão deste documento: 1.0. Documento-base para o desenvolvimento incremental do compilador MINIC.